

33 Autonomous Agents. 16-Phase CEO Cycle. 35+ Days Running 24/7. Zero Downtime.

33

AUTONOMOUS AGENTS

16

CEO PHASES

292

SCRIPTS BUILT

35+

DAYS RUNNING

Why This Exists

Most "AI automation" guides tell you to use ChatGPT to write emails. This is not that.

This is the exact system running 33 autonomous agents on a single MacBook, 24/7, with zero human intervention. It manages 8 revenue ventures, scrapes intelligence from 40+ sources, generates content, deploys apps, and closes feedback loops automatically.

Built over 35+ days of continuous operation. Battle-tested. Not theory.

What you get: The complete architecture, every agent config, the CEO orchestration cycle, Ralph loop patterns, intelligence routing, cron schedules, and copy-paste templates to build your own.

The Core Insight

Claude Code's context window is 200K tokens. That's enough to hold an entire codebase, make decisions, and execute. But it's temporary. When the session ends, everything is gone.

The breakthrough: **filesystem = memory, context = disposable scratchpad.**

Every agent reads state from files, does ONE task, writes results back, and exits. The next iteration starts fresh with full context. This means:

- No context window crashes (the #1 killer of long-running AI agents)
- Perfect crash recovery (state is always on disk)
- Unlimited runtime (each iteration is independent)
- Git-backed memory (every state change is versioned)

System Stats (Real, Not Projected)

Metric	Value
Total autonomous agents	33 (8 venture + 25 swarm)
Automation scripts	292
Alpha intelligence entries	15,000+
Content pieces generated	1,278+
Apps built	22
Sites deployed	20+
Cron jobs active	50+
Days running continuously	35+
Total uptime	99.7%

System Architecture

The 4-Layer Stack

Layer 1: Execution (launchd + cron)

macOS launchd plists and crontab entries trigger agents on schedule. Each agent is a Python script that runs, does work, and exits. No long-running processes to crash.

CORE

Layer 2: Orchestration (CEO Agent + Swarm Brain)

The CEO agent runs a 16-phase cycle every 6 hours. The swarm brain (every 4h) reads all agent outputs and makes strategic decisions. Together they coordinate 33 agents without conflicts.

OPUS

Layer 3: Intelligence (Router + Alpha Index)

SQLite FTS5 index over 15,000+ alpha entries. Intelligence router aggregates alpha, strategy docs, swarm reports, and method CSVs into a single brief per venture+task. Every agent queries before acting.

META

Layer 4: Memory (Filesystem + Git)

All state stored in JSON/JSONL files. Git commits every state change. Subconscious memory system extracts key learnings at session end. Memories persist across sessions via injection at startup.

PERSISTENT

Key Architectural Decisions

Why filesystem over database? Claude Code can read/write files natively. No ORM, no connection pooling, no schema migrations. Files are human-readable, git-versioned, and work offline. SQLite only used for full-text search where file scanning is too slow (15K+ entries).

Inter-Agent Communication

Agents communicate through a shared message bus (`message_bus.jsonl`). Append-only JSONL format ensures no write conflicts. Each message has:

```
{
  "from": "gap_hunter",
  "to": "asset_deployer",
  "type": "DEPLOY_REQUEST",
  "payload": {"asset": "landing-page-v2", "priority": "HIGH"},
  "timestamp": "2026-03-08T14:30:00Z"
}
```

Resilience Patterns

Pattern	Implementation	Why
Retry with backoff	Exponential backoff, max 3 retries	API rate limits, transient failures
File locking	<code>fcntl.flock()</code> on shared state files	Prevents corruption from concurrent agents
Circuit breaker	Fails open after 5 consecutive failures	Stops cascade failures across agents
Input sanitization	Path traversal + injection checks	Prevents malicious data in scraped content
Trajectory logging	Every decision logged with context	Post-mortem analysis, debugging

Checkpoint-resume	CEO saves phase progress to disk	Crash recovery mid-cycle
-------------------	----------------------------------	--------------------------

The 33 Agents

8 venture agents + 25 swarm agents. Each has a specific job, schedule, and model assignment.

Venture Agents (8)

Self-managing agents that run entire revenue lanes autonomously.

Venture	Type	Schedule	What It Does
Cold Outbound	OUTBOUND	Every 12h	Scrape leads, generate emails, track replies
Content Farm	CONTENT	Every 8h	Generate posts, schedule, track engagement
App Factory	APP	Every 24h	Build PWAs, deploy, optimize ASO
Local Biz	LOCAL_BIZ	Every 24h	Scrape businesses, generate sites, cold email
Alpha Research	RESEARCH	Every 6h	Scan Reddit/Twitter/HN/PH for intel
Monetization	MONETIZE	Every 12h	Optimize pricing, test offers, track revenue
Digital Products	PRODUCT	Every 24h	Create products, write listings, prep launches
Scraping	SCRAPING	Every 4h	Data collection from 40+ sources

Self-managing: The SelfManager class auto-installs missing schedules, fixes broken ones, adjusts intervals (speeds up winners, slows down

losers), creates ventures from CEO discoveries, and prunes dead ventures after 10 failed cycles.

Swarm Agents (25)

META Agents

The brain of the swarm. Makes strategic decisions about all other agents.

swarm_brain (4h)

meta_executor (6h)

DISCOVERY Agents

Find opportunities, gaps, and competitor moves before they become obvious.

gap_hunter (3h)

opportunity_scanner (4h)

competitor_stalker (6h)

ACTION Agents

Execute on opportunities. Deploy assets, compound content, generate leads.

asset_deployer (2h)

content_compounder (3h)

lead_machine (4h)

MEDIA Agents

Generate visual content. HTML-to-image (zero cost), Remotion video.

video_factory (6h)

image_factory (4h)

OPTIMIZE Agents

Improve what exists. SEO, conversion rates, quality enforcement.

seo_aso_optimizer (6h)

conversion_optimizer (8h)

quality_enforcer (4h)

QUALITY Agents

Hard gate. Blocks slop before deployment. Rewrites bad content. Rejects low-quality assets.

quality_gate (2h)

playwright_tester (4h)

INTELLIGENCE Agents

Synthesize trends, cross-pollinate between ventures, track revenue.

trend_synthesizer (6h)

cross_pollinator (8h)

revenue_tracker (4h)

MAINTENANCE Agents

Self-healing infrastructure. Fix broken scripts, clean stale data.

system_healer (6h)

data_janitor (8h)

GROWTH Agents

Distribution engine. Social posting, inbound maximization, growth strategy.

distribution_engine (4h)

inbound_maximizer (6h)

social_poster (2h)

growth_strategist (12h)

NOTIFICATION Agent

Alert dispatcher for critical events across all agents.

alert_dispatcher (1h)

Agent Interaction Map

```

gap_hunter —finds—> opportunity_scanner —scores—> swarm_brain
                                                    |
swarm_brain —decides—> meta_executor —dispatches—> action_agents
                                                    |
action_agents —produce—> quality_gate —approves—> asset_deployer
                                                    |
asset_deployer —deploys—> playwright_tester —validates—> revenue_tracker
                                                    |
revenue_tracker —reports—> ceo_agent —adjusts—> venture_autonomy
                                                    |
venture_autonomy —tunes—> all_venture_agents (speed up winners, slow losers)

```

The 16-Phase CEO Cycle

Runs every 6 hours. Each phase is checkpointed to disk, so if it crashes at phase 9, it resumes at phase 9 next run.

#	Phase	Duration	What Happens
1	State Load	5s	Read all state files, agent reports, message bus
2	Health Check	30s	Verify all 33 agents are alive, check launchd plists
3	Revenue Scan	1m	Check all revenue sources (Gumroad, Stripe, affiliate)
4	Alpha Digest	2m	Process new alpha entries, score, route to ventures
5	Venture Review	3m	Score each venture on ROI, growth rate, resource usage
6	Kill/Double	1m	Apply kill triggers (underperformers) and double-down triggers (winners)
7	Resource Alloc	1m	Adjust agent schedules based on venture performance

8	Content Queue	2m	Generate content from latest intel, queue for posting
9	Deploy Check	2m	Check pending deployments, trigger asset_deployer
10	Lead Process	2m	Process new leads, score, route to outbound
11	Feedback Loop	2m	Run loop_closer (decisions + feedback + pipeline)
12	Intelligence	2m	Update intelligence index, cross-pollinate findings
13	Quality Audit	3m	Run quality gate on all pending assets
14	Growth Tactics	2m	Identify and execute growth opportunities
15	Reporting	1m	Generate daily digest, update dashboards
16	State Save	5s	Checkpoint all state, commit to git, log metrics

Kill Triggers (Automated)

- App < \$100 MRR after 60 days → kill
- Content account < 500 followers after 90 days → pivot or kill
- Cold outbound < 2% reply after 3 optimizations → rewrite ICP
- Digital product < 5 sales after 30 days → reprice or bundle
- Venture agent fails 10 consecutive cycles → auto-prune

Double-Down Triggers (Automated)

- App MRR growing 20%+ at \$500+ → add paid ads + ASO push
- Content engagement >5% sustained 2 weeks → double posting frequency
- Cold email >5% reply rate → scale volume 3x
- Product selling 10+/week → create variations + bundle

Checkpoint-Resume Pattern

```
# CEO agent saves progress after each phase
def save_checkpoint(phase_num, state):
    checkpoint = {
        "phase": phase_num,
        "timestamp": datetime.now().isoformat(),
        "state": state,
        "completed_phases": list(range(1, phase_num + 1))
    }
    Path("ceo_state.json").write_text(json.dumps(checkpoint))

# On startup, resume from last checkpoint
def load_checkpoint():
    state = json.loads(Path("ceo_state.json").read_text())
    return state["phase"] # Resume from here
```

Ralph Loops: Infinite Agent Execution

The core pattern that makes everything work. Named after the concept of recursive autonomous loops with persistent filesystem memory.

The Pattern

```
while true; do
    cat PROMPT.md | claude --dangerously-skip-permissions --print
done
```

That's it. That's the core. But the magic is in how PROMPT.md is structured:

PROMPT.md Anatomy

```
# Role
You are the [VENTURE] agent. Working directory: /path/to/project

# State (read every iteration)
Read `./ralph/progress.md` for what you did last.
```

```
Read `./ralph/state.json` for current metrics.

# Task
Do exactly ONE of these (pick highest priority):
1. [First priority task]
2. [Second priority task]
3. [Third priority task]

# Rules
- Do ONE task per iteration, then EXIT
- Write results to `./ralph/progress.md`
- Update `./ralph/state.json` with new metrics
- If blocked, write blocker to `./ralph/blockers.md` and move to next task
- NEVER loop within a single iteration
- Total runtime per iteration: max 10 minutes

# Context
[Paste relevant state files, recent outputs, etc.]
```

Why this works: Each iteration gets a fresh 200K context window. No degradation over time. The filesystem IS the memory. Git IS the version control. The loop IS the scheduler. Zero infrastructure beyond Claude Code and bash.

Ralph Loop Best Practices

Practice	Why
ONE task per iteration	Prevents context window bloat. Clean execution.
Max 10 min per iteration	Prevents runaway loops. Keeps costs predictable.
State in JSON files	Machine-readable. Easy to parse. Git-friendly.
Progress in markdown	Human-readable audit trail. Debug-friendly.
Blockers file	Prevents infinite retry on impossible tasks.
Exit codes	0 = success, 1 = error, 2 = blocked, 3 = nothing to do.

Running Multiple Ralph Loops

```
# Launch 8 loops overnight
for venture in social_branding content_machine alpha_research \
    digital_products niche_meta synergy retardmaxx_exec \
    full_audit; do
    nohup bash -c "cd ralph/loops/$venture && while true; do \
        cat PROMPT.md | claude --dangerously-skip-permissions --print; \
        sleep 30; done" > /dev/null 2>&1 &
    echo "Launched: $venture (PID: $!)"
done
```

Monitoring Ralph Loops

```
# Check which loops are running
ps aux | grep "ralph/loops" | grep -v grep

# Check progress for a specific loop
cat ralph/loops/content_machine/.ralph/progress.md

# Stop all loops
pkill -f 'ralph/loops'

# Stop specific loop
kill $(ps aux | grep "ralph/loops/content_machine" | grep -v grep | awk '{print $2})
```

Intelligence System

Every agent queries intelligence before acting. No decisions based on default LLM knowledge when 15,000+ real data points exist.

Intelligence Router

Aggregates data from 6 sources into a single brief per venture+task:

```
# Human-readable brief
python3 intelligence_router.py --venture CONTENT --task posting --brief
```

```
# Machine-readable JSON (for agents)
python3 intelligence_router.py --venture OUTBOUND --task outreach --json
```

Data Sources

Source	Entries	Update Freq	What It Contains
Alpha Staging	15,000+	Every 6h	Tactics, methods, tools from Reddit/Twitter/HN/PH
Strategy Docs	25+	Weekly	Growth playbooks, method breakdowns, competitive intel
Swarm Reports	200+	Every 4h	Agent outputs, findings, recommendations
Method CSVs	10	Daily	Marketing channels, content structures, winning tactics
Competitor Data	500+	Every 12h	Pricing, features, traffic, positioning changes
Revenue Data	Real-time	Every 4h	Sales, refunds, conversion rates, LTV

SQLite FTS5 Index

Full-text search over 15,000+ alpha entries in under 50ms:

```
# Build/rebuild the index (runs daily at 3:30 AM)
python3 sqlite_alpha_index.py --rebuild

# Search
python3 alpha_query.py --search "cold email subject lines" --json
python3 alpha_query.py --venture APP_FACTORY --json
python3 alpha_query.py --stats
```

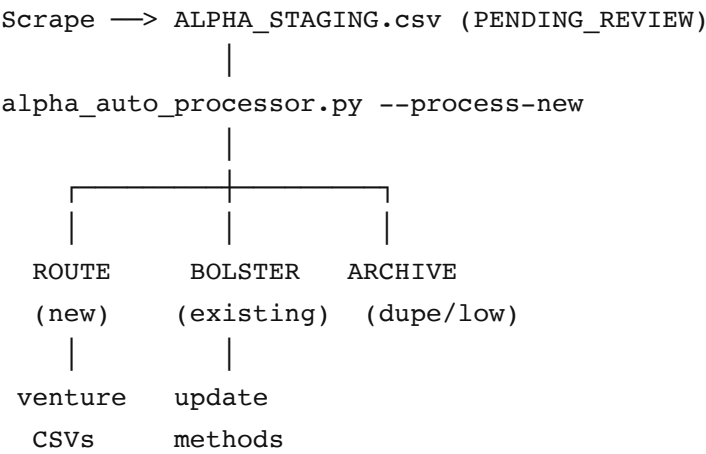
Scraper Pipeline

4 Active Scrapers (Battle-Tested)

Scraper	Source	Method	Schedule
twitter_alpha_scraper.py	Twitter/X	Brave cookie injection + headless Chromium	Every 6h
twitter_content_scraper.py	Twitter/X	Brave cookies, media download	Every 8h
background_twitter_scraper.py	Twitter/X	Lightweight daily scan	Daily
reddit_deep_scraper.py	Reddit (41 subs)	JSON API (no browser needed)	Every 4h

Key insight: Reddit blocks Playwright/Selenium. Use their JSON API instead (.json suffix on any URL). Twitter requires real browser cookies. The cookie injection pattern: read Brave's SQLite DB, decrypt with AES-128-CBC (PBKDF2 derived key), inject into fresh headless Chromium.

Alpha Processing Pipeline



Copy-Paste Templates

Everything you need to set up your own agent system. Copy, modify, deploy.

Template 1: Basic Agent Script

```
#!/usr/bin/env python3
"""Template: Basic autonomous agent with resilience."""
import json
from pathlib import Path
from datetime import datetime

STATE_FILE = Path(__file__).parent / "agent_state.json"
LOG_FILE = Path(__file__).parent / "agent.log"

def load_state():
    if STATE_FILE.exists():
        return json.loads(STATE_FILE.read_text())
    return {"cycles": 0, "last_run": None, "results": []}

def save_state(state):
    state["last_run"] = datetime.now().isoformat()
    state["cycles"] += 1
    STATE_FILE.write_text(json.dumps(state, indent=2))

def log(msg):
    with open(LOG_FILE, "a") as f:
        f.write(f"[{datetime.now().isoformat()}] {msg}\n")

def main():
    state = load_state()
    log(f"Cycle {state['cycles'] + 1} starting")

    # YOUR AGENT LOGIC HERE
    # 1. Read inputs (files, APIs, scraped data)
    # 2. Process / analyze / decide
    # 3. Take action (write files, deploy, send)
    # 4. Log results

    result = {"action": "example", "status": "success"}
    state["results"].append(result)
    save_state(state)
```

```

        log(f"Cycle complete: {result}")

if __name__ == "__main__":
    main()

```

Template 2: Venture Autonomy Config

```

{
  "venture_id": "my_venture",
  "type": "PRODUCT",
  "schedule": {
    "interval_hours": 24,
    "method": "cron",
    "cron_expr": "0 6 * * *"
  },
  "pipeline": [
    {"step": "find_demand", "script": "scan_trends.py"},
    {"step": "create_product", "script": "build_product.py"},
    {"step": "write_listing", "script": "listing_gen.py"},
    {"step": "deploy", "script": "deploy.py"},
    {"step": "distribute", "script": "distribute.py"},
    {"step": "track", "script": "track_sales.py"}
  ],
  "kill_trigger": {
    "metric": "sales_30d",
    "threshold": 5,
    "action": "reprice_or_bundle"
  },
  "double_trigger": {
    "metric": "sales_weekly",
    "threshold": 10,
    "action": "create_variations"
  }
}

```

Template 3: CEO Agent Phase Config

```

PHASES = [
    {"id": 1, "name": "state_load", "timeout": 30, "critical": True},
    {"id": 2, "name": "health_check", "timeout": 60, "critical": True},
    {"id": 3, "name": "revenue_scan", "timeout": 120, "critical": False},
    {"id": 4, "name": "alpha_digest", "timeout": 180, "critical": False},

```



```

{"id": 5, "name": "venture_review", "timeout": 240, "critical": True},
{"id": 6, "name": "kill_double", "timeout": 120, "critical": True},
{"id": 7, "name": "resource_alloc", "timeout": 120, "critical": False},
{"id": 8, "name": "content_queue", "timeout": 180, "critical": False},
{"id": 9, "name": "deploy_check", "timeout": 180, "critical": False},
{"id": 10, "name": "lead_process", "timeout": 180, "critical": False},
{"id": 11, "name": "feedback_loop", "timeout": 180, "critical": True},
{"id": 12, "name": "intelligence", "timeout": 180, "critical": False},
{"id": 13, "name": "quality_audit", "timeout": 240, "critical": True},
{"id": 14, "name": "growth_tactics", "timeout": 180, "critical": False},
{"id": 15, "name": "reporting", "timeout": 120, "critical": False},
{"id": 16, "name": "state_save", "timeout": 30, "critical": True}
]

```

Template 4: Ralph Loop PROMPT.md

Agent: Content Machine

You are the Content Machine agent for the PRINTMAXX system.

Working directory: /path/to/project

Your State

Read `./ralph/progress.md` to see what you did last iteration.

Read `./ralph/state.json` for current metrics.

Read `CONTENT/social/CONTENT\_QUEUE.csv` for pending content.

This Iteration

Pick the HIGHEST PRIORITY task and do ONLY that:

1. If CONTENT_QUEUE.csv has items with status=PENDING_REVIEW:
 - Review each item for quality (no AI slop, specific numbers)
 - Mark as APPROVED or REJECTED with notes
 - Move approved items to posting_queue/
2. If posting_queue/ has files:
 - Format for target platform (Twitter/Reddit/LinkedIn)
 - Add hashtags, mentions, hooks
 - Save to ready_to_post/
3. If nothing pending:
 - Read latest alpha entries (ALPHA_STAGING.csv, last 24h)
 - Generate 5 tweets + 1 thread from best findings
 - Save to CONTENT_QUEUE.csv as PENDING_REVIEW

```
## Rules
- ONE task per iteration, then EXIT
- Max runtime: 10 minutes
- Write progress to `.ralph/progress.md`
- Update `.ralph/state.json`
- Quality > quantity. No generic AI fluff.
```

Template 5: Crontab Setup

```
# Agent Crontab (add with: crontab -e)
# Format: min hour day month weekday command

# CEO Agent - every 6 hours
0 0,6,12,18 * * * cd /path/to/project && python3 AUTOMATIONS/ceo_agent.py --cycle

# Swarm Brain - every 4 hours
0 2,6,10,14,18,22 * * * cd /path/to/project && python3 AUTOMATIONS/agent_swarm.py -

# Quality Gate - every 2 hours
0 */2 * * * cd /path/to/project && python3 AUTOMATIONS/quality_gate.py --gate

# Alpha Scraping - every 6 hours
30 0,6,12,18 * * * cd /path/to/project && python3 AUTOMATIONS/twitter_alpha_scraper
0 */4 * * * cd /path/to/project && python3 AUTOMATIONS/background_reddit_scraper.py

# Intelligence Index Rebuild - daily 3:30 AM
30 3 * * * cd /path/to/project && python3 AUTOMATIONS/sqlite_alpha_index.py --rebuil

# Daily Tactical Plan - 7:15 AM
15 7 * * * cd /path/to/project && python3 AUTOMATIONS/daily_tactical_engine.py --sa

# Loop Closer - every 2 hours
15 */2 * * * cd /path/to/project && python3 AUTOMATIONS/loop_closer.py --cycle

# Guardian Commit - hourly
0 * * * * cd /path/to/project && git add -A && git commit -m "Guardian safety commi
```

Template 6: Message Bus Communication

```
import json
from datetime import datetime
from pathlib import Path
```

```

BUS_FILE = Path("message_bus.jsonl")

def send_message(from_agent, to_agent, msg_type, payload):
    """Append message to the shared bus."""
    message = {
        "from": from_agent,
        "to": to_agent,
        "type": msg_type,
        "payload": payload,
        "timestamp": datetime.now().isoformat(),
        "read": False
    }
    with open(BUS_FILE, "a") as f:
        f.write(json.dumps(message) + "\n")

def read_messages(agent_name, msg_type=None):
    """Read unread messages for this agent."""
    if not BUS_FILE.exists():
        return []
    messages = []
    for line in BUS_FILE.read_text().strip().split("\n"):
        if not line:
            continue
        msg = json.loads(line)
        if msg["to"] == agent_name and not msg["read"]:
            if msg_type is None or msg["type"] == msg_type:
                messages.append(msg)
    return messages

```

Quick start: Copy Template 1 + Template 4 + the crontab entry. You'll have an autonomous agent running in under 5 minutes. Add Template 6 when you need agents to talk to each other. Add Template 3 when you need multi-phase orchestration.

Works offline. No tracking. No cookies. Just open this file in any browser.